

CHAPITRE 6 : WINDOW FUNCTIONS (FONCTIONS DE FENETRAGE).

Les window functions effectuent un calcul sur un ensemble de lignes sans les regrouper. Contrairement a GROUP BY qui réduit le nombre de lignes, les window functions conservent toutes les lignes et ajoutent une colonne calculée.

Par exemple, si on veut avoir la moyenne des salaire d'une departement et les salaire de chacun de individus occupant ce departemen, ceci est possible grace au window fonctions.

Group by vs window Functions – La différence fondamentale .

Prenons par exemple concret avec 4 commandes de nos jeux de données :

commande_id	client_id	date_commande	total
1	1	2023-01-05	358.98
2	1	2023-04-20	89.99
3	2	2023-01-15	54.98
4	2	2023-06-10	119.98

Avec un Group By :

Si je veux le CA total par client :

```
SELECT client_id, SUM(total) AS ca_total  
FROM commandes  
GROUP BY client_id;
```

	client_id integer	ca_total numeric
1	1	648.94
2	2	174.96
3	3	104.97

Résultat : 1 ligne par client, les commandes individuelles sont perdues

Avec une Window Function :

Si je veux le CA par client mais aussi chaque commande :

```
SELECT commande_id, client_id, total, SUM(total)
OVER (PARTITION BY client_id) AS ca_total_client
FROM commandes;
```

	client_id integer	commande_id [PK] integer	total numeric (10,2)	ca_total_client numeric
1	1	24	199.97	648.94
2	1	1	358.98	648.94
3	1	2	89.99	648.94
4	2	3	54.98	174.96
5	2	4	119.98	174.96
6	3	25	74.98	104.97
7	3	5	29.99	104.97

Résultat : J'obtiens bien le CA par client en plus des commandes individuelles

	GROUP BY	Window Function
Lignes retournees	1 par groupe	Toutes les lignes conservees
Calcul	Agrege les lignes	Analyse les lignes
Individus	Disparaissent	Conservees avec resultat calcule
Cas d'usage	Totaux, comptes	Classements, cumuls, comparaisons

6.1 : PRINCIPES ET SYNTAXE .

Toutes les Window Functions suivent la même structure de base. Comprendre cette syntaxe une fois suffit pour utiliser toutes les fonctions.

```
fonction() OVER (
PARTITION BY colonne -- optionnel : diviser en groupes
ORDER BY colonne -- optionnel : ordre dans la fenetre
ROWS BETWEEN ... -- optionnel : bornes de la fenetre
)
```

6.1.1 – Over() – La fenêtre.

Le mot clé OVER() est ce qui transforme une fonction d'agrégation ordinaire en une window function. Sans over(), la fonction agrège et réduit les lignes. Avec Over(), elle calcul sur une fenêtre de lignes dans les fusionner.

La notion de fenêtre :

Une fenêtre (window) est un ensemble de lignes sur lequel la fonction effectue son calcul pour chaque ligne courante. (un over() vide => la fenêtre est toutes les lignes du résultat).

Exemple : Sum(total) vs sum(total) over()

```
-- Sans OVER : agregation classique -> retourne 1 seule valeur
SELECT SUM(total) FROM commandes;
-- Resultat : 1 ligne avec la somme globale
-- Avec OVER() vide : retourne la somme globale sur CHAQUE ligne
SELECT commande_id, total,
SUM(total) OVER() AS total_global
FROM commandes;
```

	commande_id [PK] integer	total numeric (10,2)	total_global numeric
1	1	358.98	2681.51
2	2	89.99	2681.51
3	3	54.98	2681.51
4	4	119.98	2681.51
5	5	29.99	2681.51

Chaque ligne conserve son identité et reçoit en plus le total global. Cela permet par exemple de calculer le pourcentage de chaque commande par rapport au CA total.

6.1.2 PARTITION BY

Partition By divise les lignes en groupes logiques pour le calcul, sans fusionner ces lignes dans le résultat. Chaque groupe est calculé indépendamment des autres ?

Comparaison group by vs partition by.

	GROUP BY client_id	PARTITION BY client_id
Lignes retournees	1 par client	Toutes les commandes conservees
Acces aux details	Non	Oui (commande_id, date, etc.)
Resultat	Reduit	Enrichi
Usage typique	Rapport agrege	Analyse ligne par ligne

Exemple : CA total par client sur chaque commande.

```
SELECT commande_id,
client_id,
date_commande,
total,
SUM(total) OVER (
PARTITION BY client_id
) AS ca_total_client
FROM commandes
ORDER BY client_id, date_commande;
```

	commande_id [PK] integer	client_id integer	date_commande date	total numeric (10,2)	ca_total_client numeric
1	1	1	2023-01-05	358.98	648.94
2	2	1	2023-04-20	89.99	648.94
3	24	1	2023-11-11	199.97	648.94
4	3	2	2023-01-15	54.98	174.96
5	4	2	2023-06-10	119.98	174.96

La fenêtre est recalculée pour chaque partition (chaque client). Pour client_id=1, la somme est 358.98 + 89.99 = 448.97. Pour client_id=2, la somme est 54.98 + 119.98 = 174.96. Les deux clients ne se mélangent pas.

A retenir : PARTITION BY divise le jeu de données en groupes logiques pour le calcul, sans fusionner les lignes. C'est la différence fondamentale avec GROUP BY.

6.1.3 : Order by dans Over().

ORDER BY dans OVER() définit l'ordre logique dans lequel les lignes sont traitées pour le calcul de la fenêtre. Il est indispensable pour les cumuls, les classements et les fonctions LAG/LEAD.

Attention : Il ne faut pas confondre ORDER BY dans OVER() et ORDER BY a la fin de la requête. Ce sont deux choses complètement différentes.

	ORDER BY dans OVER()	ORDER BY final
Role	Définit l'ordre du CALCUL	Définit l'ordre de l'AFFICHAGE
Impact	Influence le resultat numerique	Ne change pas les valeurs calculees
Exemple	Cumul dans le bon ordre	Trier le tableau final
Obligatoire	Pour cumuls, LAG, LEAD, RANK	Optionnel

Exemple :

```
SELECT commande_id,date_commande, total,  
sum(total) OVER (ORDER BY date_commande) AS ca_cumule  
FROM commandes;
```

	commande_id [PK] integer	date_commande date	total numeric (10,2)	ca_cumule numeric
1	30	2022-08-14	89.99	89.99
2	28	2022-11-05	59.99	149.98
3	29	2022-12-20	34.98	184.96
4	1	2023-01-05	358.98	543.94

L'order by date_commande dans Over() indique que le cumul doit s'effectuer en ordre chronologique. Chaque ligne reçoit la somme de toutes les lignes depuis le début jusqu'à elle-même (somme cumulée).

6.1.4 : Rows between

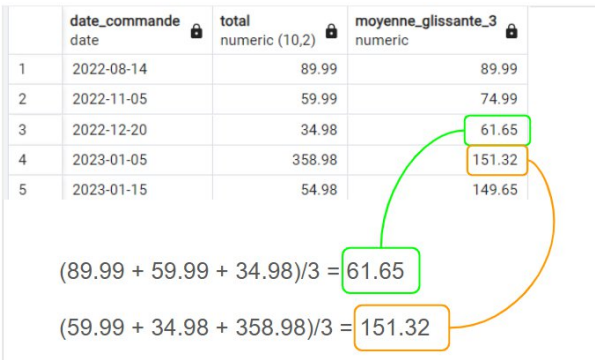
Rows between permet de définir précisément quelles lignes appartiennent à la fenêtre pour le calcul. C'est la notion de **FRAME** ou **CADRE**.

Expression	Signification
UNBOUNDED PRECEDING	Depuis la premiere ligne de la partition
CURRENT ROW	La ligne actuelle
n PRECEDING	Les n lignes precedentes
n FOLLOWING	Les n lignes suivantes
UNBOUNDED FOLLOWING	Jusqu'a la derniere ligne de la partition

Exemple d'utilisation :

Je veux calculer la moyenne des 2 lignes précédentes + ligne courante (moyenne glissante sur 3 lignes) :

```
-- Moyenne des 2 lignes precedentes + ligne courante
SELECT date_commande, total,
ROUND(AVG(total) OVER (
ORDER BY date_commande
ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
)::NUMERIC, 2) AS moyenne_glissante_3
FROM commandes
ORDER BY date_commande;
```



A RETENIR : OVER() definit une fenetre. PARTITION BY la divise en groupes. ORDER BY definit l'ordre du calcul. ROWS BETWEEN definit les bornes exactes de la fenetre.

6.2 Fonction de classement.

Comment classer des produits, clients, des ventes par ordre de performance, tout en conservant toutes les lignes dans le résultat ? => En utilisant des fonctions de classement.

Les 3 fonctions principales et leurs différence :

Exemple : Soit les produits avec les prix suivants : 100, 90, 90, 80.

prix	ROW_NUMBER()	RANK()	DENSE_RANK()
100	1	1	1
90	2	2	2
90	3	2	2
80	4	4	3

Les trois fonctions traitent différemment les ex-æquo, la différence est dans le rang attribué après les ex-æquo.

RAW_NUMBER()

Raw_number() attribue un numéro unique à chaque ligne, sans tenir compte des ex-æquo. Deux lignes ne peuvent jamais avoir le même numéro. En cas d'ex-æquo, l'ordre est arbitraire (sauf ajout d'un « tie-break »).

Exemple : classer les produits par prix :

```
SELECT nom, categorie, prix,  
ROW_NUMBER() OVER (ORDER BY prix DESC) AS rang  
FROM produits;
```

	nom character varying (150)	categorie character varying (100)	prix numeric (10,2)	rang bigint
1	Smartphone Galaxy A54	Electronique	349.99	1
2	Veste imperméable	Vetements	89.99	2
3	Clavier mécanique RGB	Informatique	89.99	3
4	Casque Bluetooth Pro	Electronique	79.99	4
5	Baskets running	Vetements	79.99	5
6	Webcam HD 1080p	Informatique	59.99	6

Observe que le Clavier mécanique et la Veste ont le même prix (89.99) mais des rangs différents (2 et 3). ROW_NUMBER() ne fait pas de partage de rang. (idem pour les rang 3 et 4 concernant des produits du même prix).

Avec PARTITION BY — Classement par catégorie

```
SELECT nom, categorie, prix,
ROW_NUMBER() OVER (
PARTITION BY categorie
ORDER BY prix DESC
) AS rang_dans_categorie
FROM produits
ORDER BY categorie, rang_dans_categorie;
```

	nom character varying (150)	categorie character varying (100)	prix numeric (10,2)	rang_dans_categorie bigint
1	Smartphone Galaxy A54	Electronique	349.99	1
2	Casque Bluetooth Pro	Electronique	79.99	2
3	Chargeur rapide 65W	Electronique	34.99	3
4	Clé USB 128Go	Electronique	19.99	4
5	Cable USB-C 2m	Electronique	8.99	5
6	Clavier mécanique RGB	Informatique	89.99	1
7	Webcam HD 1080p	Informatique	59.99	2
8	Laptop Stand aluminium	Informatique	45.99	3
9	Souris ergonomique sans fil	Informatique	29.99	4

Le classement repart à 1 pour chaque catégorie. `rang_dans_categorie = 1` identifie le produit le plus cher de chaque catégorie. Cas d'usage professionnel : filtrer uniquement les lignes avec `rang = 1` pour obtenir le top produit par catégorie.

Astuce : Pour trouver le produit le plus cher de chaque catégorie : entoure la requête dans une CTE et filtre `WHERE rang_dans_categorie = 1`.

RANK() - CLASSEMENT AVEC TROUS

`RANK()` attribue le même rang aux ex-æquo, puis saute les rangs suivants. Si deux produits sont à la 2ème place, le prochain rang sera 4 (pas 3). C'est le classement sportif dans certaines compétitions d'athlétisme, deux athlètes à ex-æquo en 2ème position sont tous les deux 2èmes, et le suivant est 4ème.

```
SELECT nom, prix,
RANK() OVER (ORDER BY prix DESC) AS rang
FROM produits;
```

	nom character varying (150)	prix numeric (10,2)	rang bigint
1	Smartphone Galaxy A54	349.99	1
2	Veste imperméable	89.99	2
3	Clavier mécanique RGB	89.99	2
4	Casque Bluetooth Pro	79.99	4
5	Baskets running	79.99	4
6	Webcam HD 1080p	59.99	6

Après les deux produits à 89.99 (rang 2), le rang suivant est 4 et pas 3. Le rang 3 est 'saute' car deux produits l'ont partagé. `RANK()` est utilisé quand les trous dans le classement ont du sens (compétitions, podiums).

DENSE_RANK() - CLASSEMENT SANS TROUS

DENSE_RANK() fonctionne comme RANK() pour les ex-aequo ? il leur attribue le meme rang mais contrairement a RANK(), il ne saute pas de rang. Le rang suivant les ex-aequo est toujours rang + 1.

```
SELECT nom, prix,  
DENSE_RANK() OVER (ORDER BY prix DESC) AS rang  
FROM produits;
```

	nom character varying (150)	prix numeric (10,2)	rang bigint
1	Smartphone Galaxy A54	349.99	1
2	Veste imperméable	89.99	2
3	Clavier mécanique RGB	89.99	2
4	Casque Bluetooth Pro	79.99	3
5	Baskets running	79.99	3
6	Webcam HD 1080p	59.99	4

Avec DENSE_RANK(), après les deux produits a 89.99 (rang 2), le suivant est 3 (pas 4). Il n'y a jamais de trous dans la numérotation. Cas d'usage : classements commerciaux, niveaux de fidélité client, segmentation sans trous.

Synthèse :

	ROW_NUMBER()	RANK()	DENSE_RANK()
Ex-aequo	Rangs differents	Meme rang	Meme rang
Trous apres ex-aequo	Non	Oui	Non
Numerotation	1,2,3,4	1,2,2,4	1,2,2,3

A RETENIR : ROW_NUMBER = toujours unique. RANK = ex-aequo + trous. DENSE_RANK = ex-aequo sans trous.

6.3 FONCTION ANALYTIQUES

Elles permettent d'analyser les données dans leur contexte temporel et relationnel : évolution dans le temps, comparaison avec les observation précédentes et suivantes, calcul de cumul ou contribution à un total.

LAG

LAG(colonne, n, défaut) retourne la valeur de la colonne n lignes avant la ligne courante dans la fenêtre définie par ORDER BY. Par défaut n=1 (ligne immédiatement précédente). Si la ligne précédente n'existe pas (première ligne), la valeur est NULL (ou la valeur par défaut si spécifiée).

Syntaxe :

```
LAG(colonne) OVER (ORDER BY ...) -- ligne precedente
LAG(colonne, 2) OVER (ORDER BY ...) -- 2 lignes avant
LAG(colonne, 1, 0) OVER (ORDER BY ...) -- ligne precedente, 0 si NULL
```

Exemple : Total et total de la commande précédente

```
SELECT commande_id,
date_commande,
total,
LAG(total) OVER (ORDER BY date_commande) AS total_precedent
FROM commandes
ORDER BY date_commande;
```

	commande_id [PK] integer	date_commande date	total numeric (10,2)	total_precedent numeric
1	30	2022-08-14	89.99	[null]
2	28	2022-11-05	59.99	89.99
3	29	2022-12-20	34.98	59.99
4	1	2023-01-05	358.98	34.98
5	3	2023-01-15	54.98	358.98

La première ligne a total_precedent = NULL car il n'y a pas de ligne avant elle. Pour la deuxième ligne (2023-01-15), total_précédent = 358.98 car la commande du 2023-01-05 la précède.

LEAD

LEAD(colonne, n, défaut) fonctionne exactement comme LAG() mais dans la direction opposée : elle retourne la valeur de la ligne n lignes après la ligne courante.

	LAG()	LEAD()
Direction	Regarde vers le PASSE	Regarde vers le FUTUR
Valeur NULL	Premiere ligne	Derniere ligne
Usage typique	Comparaison avec precedent	Comparaison avec suivant
Exemple	CA du mois precedent	Prochaine commande client

Exemple : Total et total de la commande suivante

```
SELECT commande_id,  
date_commande,  
total,  
LEAD(total) OVER (ORDER BY date_commande) AS total_suivant  
FROM commandes  
ORDER BY date_commande;
```

27	24	2023-11-11	199.97	74.98
28	25	2023-11-20	74.98	124.98
29	26	2023-12-01	124.98	44.99
30	27	2023-12-15	44.99	[null]

La dernière ligne a total_suivant = NULL car il n'y a pas de ligne après elle.

SUM() OVER()

SUM() OVER() sans PARTITION BY ni ORDER BY calcule la somme de toutes les lignes et l'affiche sur chaque ligne. C'est utile pour calculer la part de chaque ligne dans le total global.

Exemple : Pourcentage du total pour chacune des commandes.

```
SELECT commande_id,  
client_id,  
total,  
SUM(total) OVER() AS total_global,  
ROUND(total / SUM(total) OVER() * 100, 1) AS pct_du_total  
FROM commandes  
ORDER BY total DESC  
;
```

	commande_id [PK] integer	client_id integer	total numeric (10,2)	total_global numeric	pct_du_total numeric
1	1	1	358.98	2681.51	13.4
2	24	1	199.97	2681.51	7.5
3	13	10	169.97	2681.51	6.3
4	6	4	139.98	2681.51	5.2

La commande 1 représente 14.6% du CA total. Chaque ligne voit le total global identique car la fenêtre est toutes les lignes.

SUM() OVER(PARTITION BY)

Combiner SUM avec PARTITION BY permet de calculer le total de chaque groupe tout en conservant chaque ligne individuelle. C'est l'équivalent d'un GROUP BY mais sans réduire les lignes.

Exemple : Pourcentage du total des commandes pour chaque clients.

```
SELECT commande_id,  
client_id,  
total,  
SUM(total) OVER (PARTITION BY client_id) AS ca_total_client,  
ROUND(  
total / SUM(total) OVER (PARTITION BY client_id) * 100  
, 1) AS pct_du_client  
FROM commandes  
ORDER BY client_id, total DESC;
```

	commande_id [PK] integer	client_id integer	total numeric (10,2)	ca_total_client numeric	pct_du_client numeric
1	1	1	358.98	648.94	55.3
2	24	1	199.97	648.94	30.8
3	2	1	89.99	648.94	13.9
4	4	2	119.98	174.96	68.6
5	3	2	54.98	174.96	31.4

Pour client 1 : la commande de 358.98 représente 55.3% de ses dépenses totales.

Pour client 2 : la commande de 119.98 représente 68.6% de ses dépenses totales.

SUM() OVER(ORDER BY)

Quand SUM() est combine avec ORDER BY dans OVER(), elle calcule un cumul progressif : chaque ligne reçoit la somme de toutes les lignes depuis le début jusqu'à elle-même.

Exemple : CA cumulé

```
SELECT date_commande,
total,
SUM(total) OVER (
ORDER BY date_commande
ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
) AS ca_cumule
FROM commandes
ORDER BY date_commande;
```

	date_commande date	total numeric (10,2)	ca_cumule numeric
1	2022-08-14	89.99	89.99
2	2022-11-05	59.99	149.98
3	2022-12-20	34.98	184.96
4	2023-01-05	358.98	543.94
5	2023-01-15	54.98	598.92

A chaque ligne, le cumul augmente. La dernière ligne affiche le CA total. Très utilisé pour les graphiques d'évolution et les tableaux de bord financiers.

FIRST_VALUE() & LAST_VALUE()

FIRST_VALUE(colonne) retourne la valeur de la première ligne de la fenêtre.

LAST_VALUE(colonne) retourne la valeur de la dernière ligne de la fenêtre. Attention :

LAST_VALUE nécessite ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING pour voir toute la partition.

Exemple 1 : Pour chaque commande afficher aussi la date de la 1ère commande

```
SELECT
client_id,
date_commande,
FIRST_VALUE(date_commande) OVER (
PARTITION BY client_id
ORDER BY date_commande
) AS premiere_commande
FROM commandes;
```

	client_id integer	date_commande date	premiere_commande date
1	1	2023-01-05	2023-01-05
2	1	2023-04-20	2023-01-05
3	1	2023-11-11	2023-01-05
4	2	2023-01-15	2023-01-15
5	2	2023-06-10	2023-01-15

Exemple 2 : Pour chaque commande afficher aussi la date de la dernière commande

```
SELECT
  client_id,
  date_commande,
  LAST_VALUE(date_commande) OVER (
    PARTITION BY client_id
    ORDER BY date_commande
    ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING
  ) AS derniere_commande
FROM commandes;
```

	client_id integer	date_commande date	derniere_commande date
1	1	2023-01-05	2023-11-11
2	1	2023-04-20	2023-11-11
3	1	2023-11-11	2023-11-11
4	2	2023-01-15	2023-06-10
5	2	2023-06-10	2023-06-10

NTILE()

NTILE(n) divise les lignes en n groupes de taille égale et attribue à chaque ligne le numéro de son groupe (1 à n). Les lignes sont réparties dans l'ordre défini par ORDER BY.

Exemple : Répartir les produits en 4 groupes selon leur prix :

```
SELECT nom, prix,
  NTILE(4) OVER (ORDER BY prix) AS quartile
FROM produits;
```

	nom character varying (150)	prix numeric (10,2)	quartile integer
1	Cahier A5 lot x3	7.99	1
2	Cable USB-C 2m	8.99	1
3	Chaussettes lot x5	9.99	1
4	Agenda 2024	12.99	1
5	Tapis de bureau XL	19.99	1
6	Clé USB 128Go	19.99	1
7	T-shirt coton bio	19.99	1
8	Gourde isotherme 500ml	22.99	2
9	Livre SQL pour débutants	24.99	2
10	Hub USB 7 ports	24.99	2
11	Souris ergonomique sans fil	29.99	2
12	Cafetière à piston	29.99	2
13	Chargeur rapide 65W	34.99	2
14	Livre Python avancé	34.99	3
15	Coussin ergonomique	35.99	3

NTILE(4) trie les produits par prix, puis répartit les lignes en 4 groupes aussi équilibrés que possible. Le groupe 1 contient les produits les moins chers et le groupe 4 les plus chers.

A RETENIR : LAG regarde le passé. LEAD regarde le futur. SUM OVER() = total global. SUM OVER(PARTITION BY) = total par groupe. SUM OVER(ORDER BY) = cumul. NTILE(n) = division en n groupes égaux.